TOOLKIT · BACKEND ENGINEERING HANDBOOK

DOCKER COMPLETE HANDBOOK

Containers, Images, Bridge Networking, Persistent Volumes, Compose & Production Hardening

Phase	II — Backend Infrastructure
Tool ID	06 of 13
Curriculum Scope	Complete 3-Chunk Comprehensive Curriculum
Deliverable Type	Consolidated Tool Engineering Handbook (Release Build)
Source Authority	content/docker/ (01, 02, and 03 markdown sources)
Target Audience	Backend Software Engineers, SDE-1 / SDE-2, DevOps Engineers
Engineering Focus	Containerization, Multi-Stage Builds, Bridge DNS, Volumes, Compose, Hardening & CI/CD

This authoritative engineering handbook provides complete coverage of **Docker** for backend engineers. Spanning three progressive layers—Mental Models, Practical Engineering, and Production Incident Mastery—it equips developers to package Java and Spring Boot applications, manage multi-service architectures with Compose, persist state reliably using Docker volumes, configure isolated virtual bridge networks, and construct lean, secure, non-root multi-stage production container images. Designed to prepare engineers for high-concurrency production deployments and SDE technical interviews.

Curriculum Structure & Core Topics
Part I — Containers, Images & Core Mental Models: Container vs VM architecture, 5 core Docker concepts, image layering & caching, essential CLI workflows, containerizing Spring Boot, persistent named volumes, and 8-step troubleshooting workflow.
Part II — Networking, Volumes & Docker Compose: Virtual bridge networks, DNS discovery, port publishing loopbacks, volumes vs bind mounts, full-stack Compose (Spring Boot + PostgreSQL + Redis), health check dependency gates, and connection-refused debugging.
Part III — Production Images, Security, Optimization & Deployment Workflows: Multi-stage builds, non-root user enforcement, layer cache optimization, .dockerignore, graceful shutdown (SIGTERM), resource limits (CPU/cgroups), CI/CD pipelines, and memory incident diagnosis.

PART I: CONTAINERS, IMAGES & CORE MENTAL MODELS

Chunk 1 of 3 · Backend Engineering Handbook

Docker — Chunk 1: Containers, Images & Core Mental Models

Backend Engineering Tools & Infrastructure Handbook

Phase: II — Backend Infrastructure

Chunk: 1 of 3

Source file: `content/docker/01-docker-containers-images-core-mental-models.md`

Learning Objectives

By the end of this chapter, you should be able to:

- Explain why Docker exists and what problem it solves.
- Distinguish images, containers, registries, volumes, and networks.
- Explain containers versus virtual machines.
- Write a Dockerfile for a backend application.
- Build, run, inspect, and debug containers.
- Recognize common container failures and explain Docker trade-offs in SDE-1 interviews.

Layer 1 — Fundamentals & Mental Models

1. Why Docker Exists

Imagine you've built a Spring Boot application on your MacBook. It runs perfectly.

You give the project to a teammate. Their Java version differs. Their environment variables are missing. Their operating system has different dependencies. The application fails.

You deploy it to a server. It fails again.

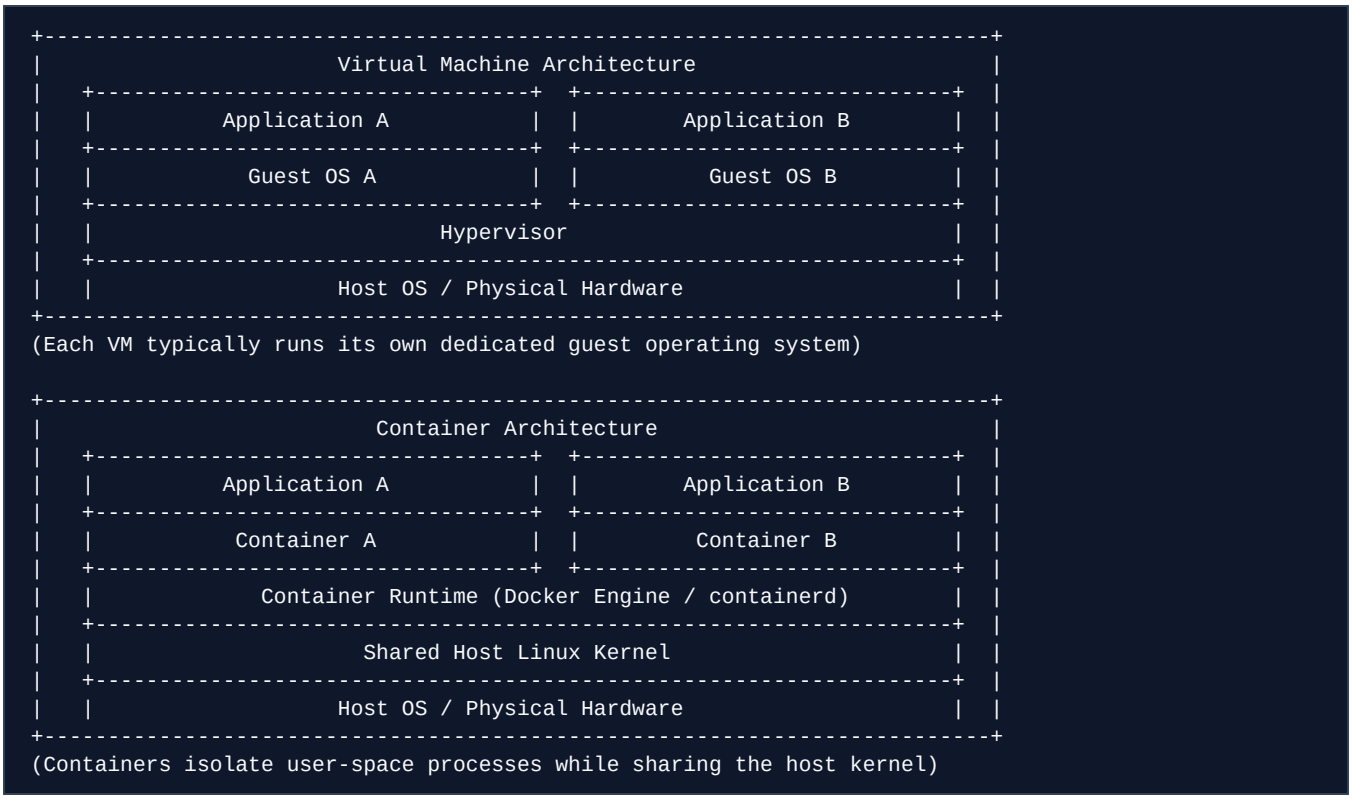
The problem is not necessarily your code. It may be the environment surrounding your code.

Docker helps package an application with its runtime and required dependencies into a standardized image, so the application can run more consistently across supported environments.

Docker does not guarantee that every environment behaves identically. Kernel differences, CPU architecture, networking, external services, configuration, and permissions still matter.

Mental model: Docker standardizes the application environment; it does not eliminate every environmental difference.

2. Containers vs Virtual Machines



Note for macOS developers: macOS does not run a Linux kernel natively. Docker Desktop on macOS transparently provisions a lightweight Linux VM under the hood to execute Linux containers.

Dimension	Virtual machine	Container
Operating system	Typically includes a guest OS	Shares the host kernel
Startup	Often slower	Often faster
Resource overhead	Usually higher	Usually lower
Isolation	Virtualized hardware and OS boundary	Process and namespace isolation with resource controls
Typical use	Full OS isolation, legacy systems	Applications, services, CI jobs

Containers are not merely lightweight VMs. Their isolation model differs, and a container should not automatically be treated as a strong security boundary against a hostile workload.

3. The Five Core Docker Concepts

1. Image

A packaged, immutable, versioned template containing the layered filesystem, application binaries, dependencies, environment defaults, and runtime metadata needed to execute containers. An image is read-only by design; a running container instantiates its own thin writable layer on top.

2. Container

An isolated, runnable user-space instance of an image with its own independent filesystem namespace, network interface, environment variables, resource limits (cgroups), and process lifecycle.

3. Registry

A centralized, secure service that indexes, stores, and distributes container images. Docker Hub is the default public registry; enterprise environments commonly deploy private registries (e.g., AWS ECR, Google Artifact Registry, GitHub Container Registry, Harbor).

4. Volume

A persistent storage mechanism managed directly by Docker outside the container's temporary union filesystem. Volumes decouple stateful data (such as PostgreSQL database directories) from the lifecycle of ephemeral container instances.

5. Network

A software-defined communication boundary (such as user-defined bridge networks) that allows containers to securely discover and communicate with one another via DNS names, or access external host and internet resources under explicit routing rules.

4. Image vs Container: The Most Important Distinction

Think of a class and an object in Java:

- A Docker image resembles a class definition in the sense that it describes what can be instantiated.
- A container resembles a running object created from that image.

The analogy is imperfect: a container is an operating-system process environment, not a Java object.

You can create multiple containers from the same image, each with separate runtime state and configuration.

```
docker pull nginx:alpine

docker run -d --name web-one nginx:alpine
docker run -d --name web-two nginx:alpine

docker ps
```

Both containers use the same image but have different names and independent container identities.

The tag `alpine` identifies a particular image tag, but tags can be updated. For strict reproducibility, production systems may pin image digests.

5. How Docker Works

At a high level:

1. You write a `Dockerfile`.
2. Docker builds an image from instructions and files.
3. The image is stored locally or pushed to a registry.
4. Docker creates a container from the image.
5. The container's main process starts.
6. Docker reports the container's state and exposes configured networking and storage.

Docker Engine commonly includes a daemon that manages images, containers, networks, and volumes, alongside a command-line client.

Docker Desktop on macOS supplies the Linux environment needed for Linux containers, since macOS does not use the Linux kernel.

6. Dockerfile and Image Layers

A `Dockerfile` is a recipe for building an image.

```
FROM eclipse-temurin:21-jre
WORKDIR /app
COPY target/app.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]
```

This example:

- Starts from a Java 21 runtime image.
- Sets `/app` as the working directory.
- Copies the application JAR into the image.
- Documents that the application uses port 8080.
- Starts the Java process when a container runs.

`EXPOSE 8080` is metadata. It does not automatically publish the port to the host.

Docker image builds use layers and caching. When an earlier layer changes, later layers may need to be rebuilt. Good Dockerfiles arrange relatively stable inputs before frequently changing application code when practical.

Layer 2 — Practical Engineering & Workflows

7. Install and Verify Docker

On your MacBook, install and start Docker Desktop, then run:

```
docker --version
docker info
docker run --rm hello-world
```

What these commands do:

- `docker --version`: displays the client version.
- `docker info`: displays engine details; it requires the daemon to be available.
- `docker run --rm hello-world`: downloads the image if needed, runs it, and removes the stopped container afterward.

If `docker info` reports that it cannot connect to the daemon, first check that Docker Desktop has started.

8. Essential Docker Commands

Command	Purpose
`docker pull nginx:alpine`	Download an image
`docker images`	List local images
`docker build -t orders-api:1.0 .`	Build an image
`docker run nginx:alpine`	Start a container in the foreground
`docker run -d --name web nginx:alpine`	Start in detached mode
`docker ps`	List running containers
`docker ps -a`	List all containers
`docker logs web`	View container logs
`docker inspect web`	Inspect configuration and state
`docker exec -it web sh`	Open a shell if the image contains one
`docker stop web`	Stop a container
`docker start web`	Start a stopped container
`docker rm web`	Remove a stopped container
`docker rmi nginx:alpine`	Remove a local image when unused

9. Run a Web Server

```
docker run -d \
  --name demo-nginx \
  -p 8080:80 \
  nginx:alpine
```

Open `http://localhost:8080`.

Understand the port mapping:

`-p 8080:80` means host port 8080 forwards to container port 80.

```
+-----------------------------------------------------+
|                      Your Mac                       |
|            Browser / curl -> localhost:8080         |
+--------------------------+--------------------------+
                           |
                           | Published port mapping (-p 8080:80)
                           v
+-----------------------------------------------------+
|                   Docker Engine                     |
|               Host 8080 -> Container 80             |
+--------------------------+--------------------------+
                           |
                           | Virtual Bridge Network
                           v
+-----------------------------------------------------+
|                  Nginx Container                    |
|            Process listens on port 80               |
+-----------------------------------------------------+
```

Check the response:

```
curl -I http://localhost:8080
docker logs demo-nginx
docker inspect demo-nginx
```

Clean up:

```
docker stop demo-nginx
docker rm demo-nginx
```

10. Containerize a Spring Boot Application

Suppose you have a Spring Boot project that produces `target/app.jar`.

Build the JAR:

```
./mvnw clean package
```

Use this Dockerfile in the project root:

```
FROM eclipse-temurin:21-jre
WORKDIR /app
COPY target/*.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]
```

Build the image:

```
docker build -t orders-api:1.0 .
```

Run the application:

```
docker run --rm \
  --name orders-api \
  -p 8080:8080 \
  orders-api:1.0
```

In a second terminal:

```
curl -i http://localhost:8080/actuator/health
```

That endpoint exists only if Spring Boot Actuator is included and configured appropriately. Otherwise, test an endpoint your application actually exposes.

Important configuration rule

If the application connects to PostgreSQL running directly on your Mac, `localhost` inside the container refers to the container itself, not your Mac.

With Docker Desktop, a container can often reach a host service using `host.docker.internal`, provided the service is listening on an address reachable from Docker and its network/security configuration permits access.

When the database runs in another container on the same Docker network, use the database container's service name as the hostname.

11. Environment Variables

Do not bake environment-specific secrets into an image.

Pass configuration at runtime:

```
docker run --rm \
  -p 8080:8080 \
  -e SPRING_PROFILES_ACTIVE=dev \
  -e DB_HOST=host.docker.internal \
  orders-api:1.0
```

This assumes your application is configured to read those variables. Spring Boot maps supported environment variables to configuration properties, but a custom property such as `DB_HOST` must be referenced or mapped appropriately.

For local experimentation, an environment file can help:

```
docker run --env-file .env orders-api:1.0
```

Keep `.env` files containing credentials out of Git. Runtime environment variables are not a complete secret-management solution: processes and privileged operators may be able to inspect them. Production deployments should use an appropriate secret-management mechanism.

12. Volumes and Data Persistence

A container's writable layer is not a reliable place to keep important data.

Create a named volume:

```
docker volume create postgres-data
docker volume ls
docker volume inspect postgres-data
```

For a local PostgreSQL container:

```
docker run -d \
  --name dev-postgres \
  -e POSTGRES_USER=app \
  -e POSTGRES_PASSWORD=local-dev-only \
  -e POSTGRES_DB=orders \
  -v postgres-data:/var/lib/postgresql/data \
  -p 5432:5432 \
  postgres:16
```

This example is for local development only. Do not use the example password in production or expose a production database without appropriate controls.

Stop and remove the container:

```
docker stop dev-postgres
docker rm dev-postgres
```

The named volume remains. Recreating the container with the same volume can preserve the database data.

Warning: Deleting a container and deleting its volume are different operations. `docker volume rm` can destroy persistent data. Back up valuable data before removing storage.

13. Inspect and Debug Containers

When a container fails, use a structured workflow.

```
docker ps -a
docker logs --tail 100 orders-api
docker inspect orders-api
docker stats
```

Ask these questions in order:

1. Was the container created successfully?
2. Is it running, exited, or restarting?
3. What does its exit status and log output show?
4. Are required environment variables present?
5. Does the process listen on the expected port?
6. Is the port published correctly?
7. Can the container resolve and reach its dependencies?
8. Are storage mounts and permissions correct?

A container can be running while its application is unhealthy. A running state alone does not prove the service is ready to accept requests.

14. Docker Cleanup

Useful commands:

```
docker container prune
docker image prune
docker system df
```

Be careful with broad cleanup commands. Pruning removes unused resources, and the exact set depends on the command and its flags. Inspect what is using disk space before removing resources.

Avoid treating `docker system prune -a --volumes` as routine cleanup on a machine containing important local data.

Layer 3 — Production Engineering & Interview Mastery

15. Common Failure Modes

Failure A: Container exits immediately

Possible causes: The main process completed, the application crashed, or the startup command was incorrect.

Investigate:

```
docker ps -a
docker logs <container>
docker inspect <container>
```

Containers typically live as long as their primary process runs. A background daemon started incorrectly may exit when the foreground process ends.

Failure B: Application starts but cannot reach PostgreSQL

Possible causes: Wrong hostname, incorrect credentials, a database that is not ready, incompatible network configuration, or firewall restrictions.

Investigate: Check application logs, database readiness, connection settings, and Docker network membership. Do not assume that `localhost` identifies another container.

Failure C: Port is already allocated

Another process or container is using the host port.

```
docker ps
lsof -i :8080
```

Choose another available host port or stop the conflicting service if appropriate.

Failure D: Image builds but the application crashes

An image build does not prove the application starts correctly.

Possible causes include a missing JAR, incorrect Java version, missing runtime configuration, invalid entrypoint, or an architecture mismatch.

Check the logs and inspect the image's build inputs and runtime configuration.

Failure E: Database data disappears

The database stored data in the container's writable layer and the container was replaced, or a volume was not mounted at the intended data path.

Use persistent storage, verify the mount, and test recovery procedures.

16. Production Image Best Practices

- Use a supported, maintained base image.
- Prefer a runtime-only image when compilation is complete outside the final image.
- Pin versions deliberately and consider image digests for reproducible deployments.
- Exclude unnecessary files using `.dockerignore`.
- Avoid copying credentials into the image.
- Run as a non-root user when practical and supported by the image.
- Keep images small enough for efficient distribution, without sacrificing required runtime dependencies.
- Scan images and keep dependencies patched.
- Use explicit health checks or orchestrator probes where appropriate.
- Send logs to standard output/error so the container platform can collect them.
- Set CPU and memory constraints at the deployment level where appropriate.
- Test graceful shutdown and signal handling.

A container image is not automatically secure just because it is small.

17. Dockerfile vs Docker Compose vs Kubernetes

Tool	Primary responsibility
Dockerfile	Defines how to build an image
Docker CLI / Engine	Builds and runs containers
Docker Compose	Defines and runs a multi-container application
Kubernetes	Orchestrates containerized workloads across a cluster

For example, a backend project may contain:

- A Spring Boot API.
- PostgreSQL for persistence.
- Redis for caching.
- Kafka for asynchronous events.

Compose can run these components together for local development. Kubernetes addresses a different set of deployment and orchestration requirements; it is not a mandatory next step for every application.

18. Fifteen Interview Questions

Q1. What problem does Docker solve?

It standardizes packaging and execution environments, improving portability and consistency across development, CI, and deployment.

Q2. What is the difference between an image and a container?

An image is the packaged template; a container is an instance created from it with runtime state.

Q3. Why are containers generally lighter than VMs?

Containers share the host kernel rather than each requiring a separate guest OS.

Q4. Does `EXPOSE 8080` publish a port?

No. It documents container port metadata. Port publishing is configured at runtime, for example with `-p 8080:8080`.

Q5. Why does a container exit when its main process exits?

The container's lifecycle is tied to its primary process. When that process finishes, the container stops.

Q6. What is a Docker volume?

A persistent storage mechanism managed by Docker that can outlive individual containers.

Q7. What is the difference between `docker stop` and `docker kill`?

`stop` requests graceful termination and then forcefully terminates after its timeout if necessary. `kill` sends a signal directly, using SIGKILL by default unless configured otherwise.

Q8. Why does `localhost` fail when one container connects to another?

Inside a container, `localhost` refers to that same container's network namespace, not another service.

Q9. What is a Docker registry?

A service for storing and distributing container images.

Q10. Why use `.dockerignore`?

It prevents unnecessary files from entering the build context, which can improve build efficiency and reduce accidental inclusion of sensitive or irrelevant files.

Q11. What is image-layer caching?

Docker can reuse unchanged build layers, making subsequent builds faster when earlier inputs remain the same.

Q12. Why should secrets not be baked into an image?

Images can be copied, cached, pushed, and inspected. Credentials embedded in them may persist and be exposed beyond their intended scope.

Q13. Does a running container guarantee a healthy application?

No. Its main process can remain alive while the service is unable to handle requests. Health checks and application-level monitoring help detect this.

Q14. When would you choose Compose over Kubernetes?

Compose is often sufficient for local multi-container development and simpler environments. Kubernetes is intended for orchestration requirements that justify its additional complexity.

Q15. How would you debug a container that works locally but fails in deployment?

Compare image versions, runtime configuration, architecture, network and port settings, resource constraints, filesystem permissions, and logs. Reproduce the deployed configuration where possible.

---

19. Hands-On Lab: Containerize and Diagnose

Complete these exercises on your local machine:

- [ ] Run `hello-world` successfully.
- [ ] Run Nginx and access it through a published port.
- [ ] Build a Docker image for a Spring Boot JAR.
- [ ] Pass runtime configuration through environment variables.
- [ ] Run PostgreSQL with a named volume.
- [ ] Remove and recreate the PostgreSQL container, then verify the volume's persistence.
- [ ] Intentionally use a wrong port and diagnose the failure.
- [ ] Intentionally configure an invalid database hostname and identify the connection failure.
- [ ] Inspect logs, container state, and configuration.
- [ ] Stop and remove the lab resources without accidentally deleting valuable data.

---

20. Mastery Checklist

- [ ] Explain the container versus VM distinction.
- [ ] Explain images, containers, registries, volumes, and networks.
- [ ] Build and run a Spring Boot image.
- [ ] Explain image layers and build caching.
- [ ] Distinguish container ports from published host ports.
- [ ] Diagnose process exits, port conflicts, and network failures.
- [ ] Explain why containers should not be the only persistence layer.
- [ ] Identify basic image-security and configuration risks.
- [ ] Explain Dockerfile, Compose, and Kubernetes responsibilities.

Final Completion Protocol

After all three Docker chunks are saved and verified:

1. Verify all three authoritative Markdown files under `content/docker/`.
2. Review the complete tool content against all three learning layers.
3. Generate one final PDF: `docs/docker/BEH_DOCKER_COMPLETE_HANDBOOK.pdf`.
4. Verify the PDF includes all three chunks, code samples, diagrams, and readable formatting.
5. Update README status based on actual verification.
6. Commit with the exact message: `Docker done`.

Do not create chunk-level PDFs. Proceed next to **Docker — Chunk 2 of 3: Networking, Volumes & Docker Compose**.

PART II: NETWORKING, VOLUMES & DOCKER COMPOSE

Chunk 2 of 3 · Backend Engineering Handbook

Docker — Chunk 2: Networking, Volumes & Docker Compose

Backend Engineering Tools & Infrastructure Handbook

Phase: II — Backend Infrastructure

Chunk: 2 of 3

Source file: `content/docker/02-docker-networking-volumes-compose.md`

Learning Objectives

By the end of this chapter, you should be able to:

- Explain Docker networking and container-to-container communication.
- Choose between bridge networks, published ports, and host connectivity.
- Persist database data using volumes and understand bind mounts.
- Define a multi-service backend using Docker Compose.
- Diagnose DNS, connection-refused, port-mapping, and persistence failures.
- Explain production trade-offs and common Docker networking interview questions.

Layer 1 — Fundamentals & Mental Models

1. Networking: Containers Need Addresses and Paths

Imagine a backend system:

- Spring Boot API listens on port `8080`.
- PostgreSQL listens on port `5432`.
- Redis listens on port `6379`.

All three services run in separate containers.

The API needs to connect to PostgreSQL and Redis. But how?

Docker provides networks that let containers communicate according to their network configuration.

Mental model: A container is an isolated process environment. To communicate, it needs a reachable destination, the correct port, and a service that is actually listening.

Three networking concepts

Concept	Meaning
Container port	Port on which the application listens inside the container
Published port	Host port mapped to a container port
Docker network	Network that allows connected containers to communicate

Suppose PostgreSQL listens on port `5432` inside its container.

If your API runs in another container on the same user-defined bridge network, it can usually connect using:

`postgres:5432`

Here, `postgres` is the service or container name resolvable on that network.

If an application runs directly on your Mac and PostgreSQL is in a container with `-p 5432:5432`, the application can typically connect using:

`localhost:5432`

The hostname changes because the API and database run in different environments.

2. Bridge Networking

Docker's bridge networking connects containers through a virtual network on the host.

A user-defined bridge network is especially useful for local multi-container applications because connected containers can discover one another by name.

Create a network:

```
docker network create backend-net
docker network ls
docker network inspect backend-net
```

Start PostgreSQL on the network:

```
docker run -d \
  --name postgres \
  --network backend-net \
  -e POSTGRES_USER=app \
  -e POSTGRES_PASSWORD=local-dev-only \
  -e POSTGRES_DB=orders \
  postgres:16
```

Start another container on the same network:

```
docker run --rm \
  --network backend-net \
  postgres:16 \
  pg_isready -h postgres -p 5432
```

This second command runs PostgreSQL's readiness utility from another container and checks whether the database is accepting connections. The database may take a short time to initialize.

Notice that we did not publish port `5432` to the host. The two containers can still communicate over their shared network.

Production principle: Publish a port only when a host-side client or external network path needs it. Container-to-container communication does not inherently require host port publishing.

3. Why `localhost` Causes So Many Bugs

`localhost` always refers to the current network environment.

> ### Where Does `localhost` Point?
>
> >
>
> - **API running directly on your Mac:** `localhost:5432` points to a service published on the Mac's host port 5432.
>
> - **API running in a container:** `localhost:5432` points to port 5432 inside the API container itself (loopback network namespace).
>
> - **API container connecting to Compose PostgreSQL:** `postgres:5432` points to PostgreSQL's service name and internal container port on the shared bridge network.

If your API container tries to connect to `localhost:5432`, it will not automatically reach the PostgreSQL container.

The hostname must reflect where the destination actually runs.

4. Port Publishing

Consider:

```
docker run -d \
  --name web \
  -p 8080:80 \
  nginx:alpine
```

The mapping is:

`host port 8080` → `container port 80`

The browser reaches `http://localhost:8080`, while Nginx listens on port 80 inside the container.

If the application listens on port `8080` inside a Spring Boot container, a common mapping is:

```
docker run -p 8080:8080 orders-api:1.0
```

For a local development service that should only be reachable through the host loopback interface, you can bind explicitly:

```
docker run -p 127.0.0.1:8080:8080 orders-api:1.0
```

Publishing ports can make services reachable beyond the intended audience depending on the host, firewall, Docker configuration, and network environment. Avoid exposing databases or administrative services unnecessarily.

5. Volumes vs Bind Mounts

Containers have writable layers, but that layer is tied to the container's lifecycle. Persistent data should live outside that layer.

Docker supports two important storage approaches.

Property	Named volume	Bind mount
Source	Managed by Docker	Specific host path
Typical use	Database data and persistent application state	Source code, local configuration, development files
Host-path control	Docker manages storage location	You choose the path
Portability	Easier to reference by volume name	Depends on host directory layout

Create a named volume:

```
docker volume create pgdata
```

Mount it into PostgreSQL:

```
docker run -d \
  --name postgres \
  -e POSTGRES_PASSWORD=local-dev-only \
  -v pgdata:/var/lib/postgresql/data \
  postgres:16
```

Remove and recreate the container without deleting the volume, and the database data can persist.

A bind mount is useful when you want edits on your Mac to be visible inside a development container:

```
docker run --rm \
  -v "$PWD:/workspace" \
  -w /workspace \
  alpine:3.20 \
  ls -la
```

This mounts the current directory at `/workspace`. Host filesystem permissions and path behavior still matter.

Important distinction

A volume provides persistence, not a complete backup strategy.

For valuable data, plan and test backups, recovery, and retention. A volume can still be deleted, corrupted, or lost with its underlying storage.

6. Docker Compose: One Definition, Multiple Services

A real backend often depends on more than one process.

You might need:

- Spring Boot for HTTP requests.
- PostgreSQL for durable relational data.
- Redis for caching.
- Kafka for asynchronous events.

Starting each service manually is error-prone. Docker Compose lets you define services, networks, volumes, and configuration in a YAML file.

Mental model: Compose describes a multi-container application and provides commands to manage its lifecycle.

Compose is useful for local development, testing, and some deployments. It does not automatically provide every production orchestration feature.

Layer 2 — Practical Engineering & Workflows

7. Build a Multi-Service Application with Compose

Create a project structure:

```
orders-platform/
■■■ compose.yaml
■■■ .env
■■■ .gitignore
■■■ api/
    ■■■ Dockerfile
    ■■■ ...
```

The following Compose file starts PostgreSQL and Redis:

```
services:
  postgres:
    image: postgres:16
    environment:
      POSTGRES_DB: orders
      POSTGRES_USER: app
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD:?Set POSTGRES_PASSWORD}
    volumes:
      - postgres-data:/var/lib/postgresql/data
    ports:
      - "127.0.0.1:5432:5432"
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U app -d orders"]
      interval: 5s
      timeout: 3s
      retries: 10

  redis:
    image: redis:7-alpine
    command: ["redis-server", "--appendonly", "yes"]
    volumes:
      - redis-data:/data
    ports:
      - "127.0.0.1:6379:6379"
    healthcheck:
      test: ["CMD", "redis-cli", "ping"]
      interval: 5s
      timeout: 3s
      retries: 10

volumes:
  postgres-data:
  redis-data:
```

This example is for local development. The published ports bind to loopback, and the database password must be supplied through configuration. In production, use managed secret handling and an appropriately secured network.

The Redis append-only file setting enables Redis persistence behavior, but the durability guarantee depends on Redis configuration and the underlying storage.

Create a local `.env` file:

```
POSTGRES_PASSWORD=replace-with-a-local-development-password
```

Add `.env` to `.gitignore`:

```
.env
```

Do not commit real credentials. Compose interpolation can be used to keep values out of the YAML file, but an `.env` file is not itself a secure secret manager.

8. Start and Manage Services

Run these commands from the directory containing `compose.yaml`:

```
# Validate the resolved Compose configuration
docker compose config

# Start services in the background
docker compose up -d

# List service status
docker compose ps

# Follow logs
docker compose logs -f

# View logs for one service
docker compose logs -f postgres

# Stop and remove containers and network
docker compose down
```

The named volumes remain after a normal `docker compose down`.

Be careful with:

```
docker compose down -v
```

The `-v` flag removes Compose-managed named volumes declared for the project and attached anonymous volumes. That can permanently remove local database data.

9. Add the Spring Boot API

Assume your application image is built from the `api/` directory and listens on port `8080`.

Add this service to the same Compose file:

```
  api:
    build:
      context: ./api
    ports:
      - "127.0.0.1:8080:8080"
    environment:
      SPRING_DATASOURCE_URL: jdbc:postgresql://postgres:5432/orders
      SPRING_DATASOURCE_USERNAME: app
      SPRING_DATASOURCE_PASSWORD: ${POSTGRES_PASSWORD:?Set POSTGRES_PASSWORD}
      SPRING_DATA_REDIS_HOST: redis
      SPRING_DATA_REDIS_PORT: 6379
    depends_on:
      postgres:
        condition: service_healthy
      redis:
        condition: service_healthy
```

The exact Spring configuration property names depend on your application and Spring Boot version. Confirm that the app uses the properties shown.

`depends_on` with `service_healthy` can delay startup until the configured health checks pass. It does not guarantee that every later dependency operation will succeed. Applications should still handle connection failures and reconnect where appropriate.

Start the stack:

```
docker compose up -d --build
docker compose ps
docker compose logs -f api
```

Test an endpoint your application actually exposes:

```
curl -i http://localhost:8080/actuator/health
```

This endpoint requires Spring Boot Actuator and suitable configuration. If your app does not have Actuator, use one of its actual API routes.

Key networking insight

From the API container:

- PostgreSQL is reached through **`postgres:5432`**.
- Redis is reached through **`redis:6379`**.

These names resolve through the Compose network. You do not need to publish PostgreSQL or Redis ports merely for the API container to reach them.

10. Development Workflow with Bind Mounts

A development container may mount source code from the host to support rapid iteration.

For example:

```
services:
  api:
    build:
      context: ./api
    volumes:
      - ./api:/workspace
    working_dir: /workspace
```

This configuration only mounts the source. It does **not** automatically compile the project, reload the JVM, or run the application from the mounted source. You must configure the container command and development tooling to achieve those behaviors.

For production images, copying a built artifact into the image is often more predictable than mounting an entire source tree.

11. Practical Debugging Commands

```
# Container and service status
docker compose ps
docker ps -a

# Logs
docker compose logs --tail 100 api
docker compose logs --tail 100 postgres

# Resolved configuration
docker compose config

# Network details
docker network ls
docker network inspect <network-name>

# Volume details
docker volume ls
docker volume inspect <volume-name>

# Resource usage
docker stats
docker system df
```

For a service that cannot connect to PostgreSQL, investigate:

1. Is PostgreSQL running?
2. Is its health check passing?
3. Is the API attached to the same Compose network?
4. Is the API using the service name `postgres`, not `localhost`?
5. Are credentials and database names correct?
6. Is PostgreSQL actually ready to accept connections?
7. What does the API log say about the failure?

Layer 3 — Production Engineering & Interview Mastery

12. Diagnose Common Networking Failures

Scenario A: `Connection refused`

This usually means the destination was reached but no process accepted the connection at that address and port, or a network rule actively rejected it.

Check whether the service is running, listening on the expected port, and reachable from the caller's network environment.

Scenario B: `Could not resolve host`

The hostname could not be resolved in the caller's environment.

Check the hostname, Docker network membership, and whether the intended service name is available on that network.

Scenario C: The API connects locally but fails in Compose

A frequent cause is a configuration value such as:

```
SPRING_DATASOURCE_URL=jdbc:postgresql://localhost:5432/orders
```

Inside the API container, `localhost` points back to the API container. Use the Compose service name for a database container on the same network:

```
SPRING_DATASOURCE_URL=jdbc:postgresql://postgres:5432/orders
```

Scenario D: The database loses data after a restart

Determine whether the container was merely restarted or removed and recreated, and verify that the intended persistent volume is mounted at the database's actual data directory.

A restart and a recreation are not the same lifecycle operation.

13. Production Trade-offs and Security

Network exposure

- Avoid publishing database and cache ports when only internal services require access.
- Bind development ports to loopback when appropriate.
- Use authentication and network controls for services that support them.
- Do not assume that an internal Docker network alone satisfies every production security requirement.

Persistent storage

- Named volumes are useful for local stateful services.
- Backups must be tested independently of the volume.
- Understand the data-loss implications of volume deletion.
- For production, choose storage based on durability, availability, recovery objectives, and operational ownership.

Configuration

- Keep environment-specific configuration outside immutable application images.
- Avoid committing credentials.
- Use a proper secret-management approach in production.
- Validate configuration at startup and make missing required settings obvious.

Startup and resilience

- A dependency being started does not guarantee it is ready.
- Health checks help detect some readiness failures.

- Applications should tolerate transient dependency failures where appropriate.
- Avoid infinite restart loops that conceal a persistent configuration error.

Compose versus orchestration

Compose is excellent for local environments and integration testing. For larger production environments, teams may need orchestration, rolling deployment controls, service discovery, resource policies, and automated recovery. Kubernetes can address many of these needs, but its operational complexity should be justified by the requirements.

14. Fifteen Interview Questions

Q1. What is a Docker bridge network?

A virtual network that connects containers. User-defined bridge networks support service-name-based communication between connected containers.

Q2. Why should a containerized API not normally use `localhost` for another container?

Because `localhost` refers to the API container itself, not the other service.

Q3. What does `-p 8080:80` mean?

It maps host port `8080` to container port `80`.

Q4. Do two containers need published ports to communicate?

No. Containers on a suitable shared Docker network can communicate through their internal ports.

Q5. What is the difference between a volume and a bind mount?

A volume is managed by Docker; a bind mount maps a specific host path into a container.

Q6. Why use named volumes for databases?

They allow data to persist independently of an individual container's writable layer and make storage references easier to manage.

Q7. Does a named volume guarantee backups?

No. It provides persistence, not a backup and recovery strategy.

Q8. What does Docker Compose solve?

It defines and manages a multi-service application, including services, networks, volumes, and configuration.

Q9. What is the role of `depends_on`?

It expresses service dependencies and can control startup ordering. With supported health-check conditions, it can wait for a dependency to become healthy, but it does not eliminate runtime failures.

Q10. What does a health check establish?

It runs a configured test and reports health status according to its result. A poorly designed health check may still miss important application failures.

Q11. Why can a database work from the host but not from an API container?

The hostname, port mapping, network membership, credentials, or service readiness may differ between those environments.

Q12. What does `docker compose down -v` do?

It removes the Compose project's containers and network and removes declared named volumes and attached anonymous volumes. Persistent data may be lost.

Q13. When should you publish a container port?

When a client outside the container's internal network needs to reach that service through the host's published port.

Q14. Why is an `.env` file not sufficient production secret management?

It is a configuration convenience, not a complete access-control, auditing, rotation, or secret-distribution system.

Q15. How would you debug an API-to-database connectivity incident?

Inspect service health and logs, verify hostname and port, check network membership, validate credentials and configuration, and test connectivity from the caller's environment.

---

15. Hands-On Capstone

Build a local backend environment with:

- Spring Boot API
- PostgreSQL
- Redis
- Docker Compose
- Persistent named volumes
- Health checks for dependencies

Complete the following:

- [ ] Create `compose.yaml`.
- [ ] Configure a local `.env` file and ignore it in Git.
- [ ] Start PostgreSQL and Redis.
- [ ] Verify both health checks.
- [ ] Add the Spring Boot API service.
- [ ] Configure internal service-name-based connections.
- [ ] Verify the API can reach its dependencies.
- [ ] Stop and recreate the database container and verify data persistence.
- [ ] Break the database hostname intentionally and diagnose the failure.
- [ ] Remove the lab resources carefully without deleting data unintentionally.

---

16. Mastery Checklist

- [ ] Explain container ports versus published host ports.
- [ ] Explain bridge networks and container DNS.
- [ ] Distinguish named volumes from bind mounts.
- [ ] Explain why `localhost` behaves differently inside a container.
- [ ] Write a Compose file for a backend and its dependencies.
- [ ] Use health checks and understand their limitations.

- [ ] Debug DNS, refused connections, and dependency startup failures.
- [ ] Explain persistent storage and backup trade-offs.
- [ ] Secure ports, configuration, and credentials appropriately.

Final Completion Protocol

After all three Docker chunks are saved and verified:

1. Verify all three source files under `content/docker/`.
2. Review the complete content against the three learning layers.
3. Generate one final PDF at `docs/docker/BEH_DOCKER_COMPLETE_HANDBOOK.pdf`.
4. Verify the PDF contains all three chunks, commands, diagrams, and readable formatting.
5. Update README status using verified progress.
6. Commit with the exact message:

```
Docker done
```

Do not generate a separate PDF for this chunk.

Next: **Docker — Chunk 3 of 3: Production Images, Security, Optimization & Deployment Workflows**.

PART III: PRODUCTION IMAGES, SECURITY, OPTIMIZATION & DEPLOYMENT WORKFLOWS

Chunk 3 of 3 · Backend Engineering Handbook

Docker — Chunk 3: Production Images, Security, Optimization & Deployment Workflows

Backend Engineering Tools & Infrastructure Handbook

Phase: II — Backend Infrastructure

Chunk: 3 of 3

Source file: `content/docker/03-docker-production-security-optimization-deployment.md`

Learning Objectives

By the end of this chapter, you should be able to:

- Build production-oriented images using multi-stage Dockerfiles.
- Reduce image size and improve build reproducibility.
- Apply practical container security controls.
- Understand resource limits, health checks, graceful shutdown, and restart behavior.
- Integrate Docker into a CI/CD deployment workflow.
- Diagnose production container incidents and defend engineering trade-offs in SDE-1 interviews.

Layer 1 — Fundamentals & Mental Models

1. Development Images vs Production Images

A development environment prioritizes convenience. A production image prioritizes reliable operation, security, reproducibility, and efficient deployment.

A development image might contain:

- Source code and build tools.
- Maven and a JDK.
- Debugging utilities.

- Hot-reload tooling.
- Local configuration.

A production runtime image usually needs:

- The application artifact.
- The compatible runtime.
- Required runtime libraries and certificates.
- An explicit startup command.
- Appropriate permissions and configuration.

Mental model: Build the application in an environment designed for building it; run it in an environment designed for running it.

This separation reduces unnecessary software in the final image and limits the number of tools available to an attacker if the container is compromised.

2. Multi-Stage Builds

A multi-stage Dockerfile uses multiple `FROM` instructions. Artifacts produced in one stage can be copied into a later stage.

For a Spring Boot application:



Benefits include:

- Build tools are not required in the final runtime image.
- The final image can be smaller.
- The build and runtime environments have clearer responsibilities.
- The final image contains fewer unnecessary components.

Multi-stage builds do not automatically make an image secure or reproducible. Base-image selection, dependency management, permissions, secrets, and build inputs still matter.

3. Image Size, Layers, and Build Cache

Docker builds images from instructions and filesystem layers. Cached layers can be reused when their relevant inputs have not changed.

Consider two approaches.

Less cache-friendly:

```
COPY . .
RUN ./mvnw package
```

More deliberate:

```
COPY .mvn/ .mvn/
COPY mvnw pom.xml ./
RUN chmod +x mvnw && ./mvnw -B dependency:go-offline

COPY src/ src/
RUN ./mvnw -B package
```

The second approach separates dependency inputs from frequently changing source files, so source edits may not invalidate the dependency-download layer.

This is a general pattern rather than a guarantee. Maven plugins, profiles, parent POMs, generated sources, and other build inputs may require additional files to be copied before the dependency layer.

What makes images large?

- Unnecessary build tools and files.
- Large base images.
- Build caches copied into the runtime image.
- Logs, test output, or local development files included in the build context.
- Redundant layers and downloaded dependencies.

A smaller image can reduce transfer time and storage consumption. However, aggressively minimizing an image can make troubleshooting harder or remove required runtime components.

4. Container Security: Defense in Depth

A container should not be treated as a security boundary that automatically makes a workload safe.

A practical security model includes several controls:

Control	Why it matters
Minimal maintained base image	Reduces unnecessary packages and exposure
Non-root execution	Limits privileges available to the application process
No embedded secrets	Prevents credentials from persisting in image layers
Image scanning	Helps identify known vulnerabilities
Read-only filesystem where feasible	Reduces unintended filesystem modifications
Resource limits	Restricts resource consumption
Least-privilege capabilities	Limits privileged kernel operations

Control	Why it matters
Trusted image provenance	Improves confidence in the image's origin

No single control is sufficient. A non-root process can still exploit application vulnerabilities, leak accessible secrets, or exhaust resources within its limits.

5. Container Lifecycle and Health

A container has a lifecycle, and its main process is central to that lifecycle.

Important concepts:

- **Running:** The container's main process is running.
- **Exited:** The main process has stopped.
- **Restart policy:** Determines whether Docker should restart the container under configured conditions.
- **Health check:** Reports whether a configured test considers the container healthy.
- **Graceful shutdown:** Allows the application to stop accepting new work and finish or safely abandon in-flight work.

A running container is not necessarily a healthy service.

A restart policy is also not a substitute for fixing a bad deployment. An application that crashes due to invalid configuration may simply enter a repeated restart cycle.

Layer 2 — Practical Engineering & Workflows

6. Build a Production-Oriented Spring Boot Image

Assume the repository contains a standard Maven Spring Boot application.

Use this multi-stage Dockerfile as a starting point:

```
# syntax=docker/dockerfile:1

FROM eclipse-temurin:21-jdk AS build
WORKDIR /workspace

COPY .mvn/ .mvn/
COPY mvnw pom.xml ./
RUN chmod +x mvnw \
    && ./mvnw -B dependency:go-offline

COPY src/ src/
RUN ./mvnw -B clean package -DskipTests

FROM eclipse-temurin:21-jre
WORKDIR /app

RUN groupadd --system app \
    && useradd --system --gid app app

COPY --from=build --chown=app:app \
    /workspace/target/*.jar /app/app.jar

USER app:app
EXPOSE 8080

ENTRYPOINT ["java", "-jar", "/app/app.jar"]
```

This example assumes:

- A Linux base image with the expected user-management utilities.
- A Java 21 application.
- A Maven Wrapper and conventional source layout.
- A single runnable JAR in `target/`.
- The application listens on port 8080.

If your project has multiple JARs, generated sources, a custom Maven layout, or additional build inputs, adapt the Dockerfile accordingly.

Important build-quality decision

The example uses `-DskipTests` in the image build stage. This can be appropriate when a separate CI stage has already run the tests and the pipeline enforces that gate.

It is **not** appropriate to use this flag as a way to bypass verification without another test stage.

A CI pipeline should run tests before publishing the image.

More reproducible builds

For stronger reproducibility:

- Pin base images to reviewed versions or digests.
- Use the Maven Wrapper with a known Maven version.
- Keep dependencies controlled and review updates.
- Avoid downloading unreviewed artifacts during deployment.
- Record the source revision and image digest.

- Make build configuration explicit.

A versioned tag is useful, but a mutable tag can point to a different image later. A digest identifies a specific image manifest.

7. Create a `.dockerignore`

Example:

```
.git
.idea
.vscode
.DS_Store

target/
build/
out/

.env
.env.*
!.env.example

*.log
coverage/
```

This reduces the build context and prevents many unnecessary local files from being sent to the builder.

Review exceptions carefully. If the build requires a generated artifact from `target/`, excluding that directory is correct only when the Dockerfile builds the artifact internally or obtains it through another explicit mechanism.

Do not include real credentials in `.env.example`.

8. Add a Docker Health Check

A health check can detect some application failures:

```
HEALTHCHECK --interval=30s \
            --timeout=5s \
            --start-period=40s \
            --retries=3 \
  CMD ["sh", "-c", "/* replace with a real health-check command */"]
```

The command above is a **placeholder**, not a working health check. Replace it with a command that exists in your image and actually tests the intended readiness or health condition.

For example, if the runtime image contains `curl` and the application exposes a suitable Actuator health endpoint:

```
HEALTHCHECK --interval=30s \
            --timeout=5s \
            --start-period=40s \
            --retries=3 \
  CMD curl -fsS http://localhost:8080/actuator/health || exit 1
```

That version works only if `curl` is installed, Actuator is included, and the endpoint is accessible as configured. Minimal runtime images often omit `curl`; installing a utility solely for a health check increases the image footprint.

Docker health status does not automatically restart a container merely because it becomes unhealthy. Restart policies primarily respond to container termination, not to the health status alone. Compose, orchestrators, and monitoring systems may use health information in their own ways.

9. Configure Graceful Shutdown

A backend may receive a termination signal during a deployment or container stop.

The application should be prepared to:

1. Stop accepting new work.
2. Finish in-flight requests within a bounded time.
3. Close connections and release resources.
4. Exit before the termination deadline when possible.

For Spring Boot, graceful shutdown can be configured through application properties, subject to the version and server configuration:

```
server.shutdown=graceful
spring.lifecycle.timeout-per-shutdown-phase=20s
```

Set the Docker stop timeout to accommodate the shutdown behavior:

```
docker run -d \
  --name orders-api \
  --stop-timeout 30 \
  -p 8080:8080 \
  orders-api:1.0
```

The application, entrypoint, and runtime must handle termination signals correctly. A shell wrapper that fails to forward signals can interfere with graceful shutdown; exec-form `ENTRYPOINT` helps avoid this common problem.

10. Resource Limits

Containers can consume CPU, memory, disk, and other host resources.

For a local Docker run:

```
docker run -d \
  --name orders-api \
  --memory=512m \
  --cpus=1.0 \
  -p 8080:8080 \
  orders-api:1.0
```

This limits the container to a configured memory limit and CPU quota.

The appropriate values depend on the application, workload, JVM configuration, and deployment environment. A limit that is too low can cause latency spikes, memory failures, or out-of-memory termination.

Inspect usage:

```
docker stats
docker inspect orders-api
```

For Java applications, container-aware JVM behavior depends on the Java version and JVM flags. Measure actual usage and configure heap sizing with awareness of memory required outside the Java heap, including thread stacks, direct buffers, metaspace, and other native allocations.

11. Image Build and Verification Workflow

A disciplined local workflow:

```
# Run application tests
./mvnw clean verify

# Build the image
docker build -t orders-api:1.0 .

# Inspect image size and metadata
docker images orders-api
docker image inspect orders-api:1.0

# Start the container
docker run -d \
  --name orders-api \
  -p 8080:8080 \
  orders-api:1.0

# Inspect the runtime
docker ps
docker logs --tail 100 orders-api

# Stop and clean up the container
docker stop orders-api
docker rm orders-api
```

`verify` runs the Maven lifecycle through the verification phase, subject to the project's configured plugins and lifecycle behavior.

Do not publish the image merely because `docker build` succeeds. Build success establishes that the image could be constructed; it does not prove that the application starts, is secure, or functions correctly.

12. CI/CD Workflow

A practical deployment pipeline can follow this sequence:

1. Checkout a specific source revision.
2. Install or select the required JDK.
3. Run unit and integration tests.
4. Build the application artifact.
5. Build the Docker image.
6. Scan dependencies and the image.
7. Publish the image to a registry if checks pass.
8. Deploy the exact image version or digest.
9. Verify service readiness and key application behavior.
10. Monitor the rollout and roll back if required.

Example shell sequence for a pipeline that already has registry authentication configured:

```
set -euo pipefail

./mvnw -B clean verify

docker build \
  -t registry.example.com/orders-api:"$GIT_COMMIT" \
  .

docker push \
  registry.example.com/orders-api:"$GIT_COMMIT"
```

This is an illustrative sequence. A real CI environment must define `GIT_COMMIT`, configure the registry, protect credentials, and enforce the necessary checks.

Avoid deploying an image solely by a mutable tag such as `latest`. Record which immutable image digest is deployed so you can identify and reproduce the exact version.

Layer 3 — Production Engineering & Interview Mastery

13. Production Incident Scenarios

Incident A: The new image is much larger than expected

Possible causes:

- The build stage was accidentally used as the final stage.
- Local files or generated artifacts entered the build context.
- The runtime image contains unnecessary packages.
- Build caches or dependency directories were copied into the final image.

Response:

1. Compare the previous and current image sizes.
2. Inspect the Dockerfile stages.
3. Review `.dockerignore`.
4. Inspect image layers and their sizes.
5. Verify that the final stage copies only required runtime artifacts.

Incident B: Container memory usage rises until the service crashes

Possible causes:

- Increased request concurrency or workload.
- A memory leak or unbounded cache.
- Oversized payloads or queues.
- JVM heap settings that leave too little room for native memory.

- An overly restrictive container limit.

Response:

1. Confirm whether the container was terminated and inspect its state.
2. Review application logs and memory metrics.
3. Compare heap and total container memory.
4. Inspect workload and traffic changes.
5. Adjust limits or application behavior based on evidence.
6. Load-test the revised configuration.

Do not assume every memory-related restart is a Java heap leak.

Incident C: Deployment succeeds but the service is unavailable

Possible causes:

- Incorrect runtime environment variables.
- A container that is running but not ready.
- A port or network configuration mismatch.
- A database migration or dependency failure.
- A failed health check.
- A broken image entrypoint.

Response:

- Inspect container state and logs.
- Verify the deployed image digest.
- Check the actual listening port and published mapping.
- Test dependency connectivity.
- Verify readiness and application-level health.
- Roll back if the deployment violates the service's recovery criteria.

Incident D: A container restarts repeatedly

Possible causes:

- Invalid configuration.
- Application startup exception.
- Missing credentials.
- Failed dependency initialization.
- Memory pressure or an explicit process exit.

Response:

```
docker ps -a
docker logs --tail 200 <container>
docker inspect <container>
docker events
```

Identify the initial failure before changing restart settings. Repeated restarts can increase load on dependencies and make diagnosis more difficult.

14. Production Readiness Checklist

Before shipping an image, verify:

- [ ] The base image is maintained and its version is controlled.
- [ ] The build uses explicit, reviewed dependencies.
- [ ] The final image excludes unnecessary build tooling.
- [ ] The application runs with minimal practical privileges.
- [ ] Secrets are not embedded in image layers.
- [ ] The image has been scanned according to the project's security policy.
- [ ] Configuration is supplied explicitly at runtime.
- [ ] Health checks reflect the intended health condition.
- [ ] Graceful shutdown behavior is tested.
- [ ] Resource limits are appropriate for expected workload.
- [ ] Logs are available through the container platform.
- [ ] The image version or digest is recorded.
- [ ] CI tests pass before publication.
- [ ] Rollback and incident diagnosis procedures are understood.

15. Fifteen Interview Questions

Q1. Why use a multi-stage Docker build?

To separate the build environment from the runtime image and copy only required artifacts into the final stage.

Q2. What is the difference between a JDK and a JRE?

A JDK includes development tools such as a compiler. A runtime environment provides what is needed to execute Java applications. Modern Java distributions and packaging can differ, so select a compatible supported image.

Q3. Why should containers run as non-root?

It reduces the privileges available to a compromised application process, limiting some potential damage.

Q4. What is `.dockerignore`?

A file that excludes specified paths from the Docker build context.

Q5. What is Docker layer caching?

Docker reuses unchanged build layers where possible, reducing repeated build work.

Q6. Does a smaller image automatically mean a safer image?

No. Security also depends on vulnerabilities, configuration, permissions, provenance, and runtime exposure.

Q7. What is the difference between a container's running state and its health status?

Running describes the main process state; health status reflects the result of a configured health check.

Q8. Does Docker automatically restart an unhealthy container?

Not merely because it is unhealthy. Restart policies primarily act when the container terminates. Additional orchestration or monitoring behavior may be needed.

Q9. Why use exec-form `ENTRYPOINT`?

It starts the application as the main process rather than placing it behind a shell wrapper that may interfere with signal handling.

Q10. Why configure a stop timeout?

To give the application time to shut down gracefully before forced termination.

Q11. Why can Java memory usage exceed the heap limit you expected?

The container's memory usage includes native allocations, thread stacks, direct buffers, metaspace, and other processes or overhead—not just the Java heap.

Q12. Why deploy by image digest?

A digest identifies a specific image manifest and avoids ambiguity from mutable tags.

Q13. What should a container health check test?

A condition that meaningfully reflects the intended health or readiness state, without being so broad or fragile that it creates misleading results.

Q14. How should CI handle image publication?

Run required tests and security checks first, authenticate safely to the registry, publish a versioned image, and record the resulting artifact identity.

Q15. How do you troubleshoot a failed production container deployment?

Inspect the deployed image, container state, logs, environment configuration, networking, resource limits, and health status. Compare against the last known-good release and roll back when required.

---

16. Hands-On Capstone: Ship a Spring Boot Container

Use a small Spring Boot API and complete the following:

- [ ] Create a multi-stage Dockerfile.
- [ ] Add a `.dockerignore`.
- [ ] Build and test the application.
- [ ] Run the application as a non-root user.
- [ ] Configure environment-specific values outside the image.
- [ ] Add a meaningful health check if supported by the image.
- [ ] Test graceful shutdown.
- [ ] Apply a memory limit and observe resource usage.
- [ ] Inspect the image metadata and size.
- [ ] Run a simulated failure and diagnose it from logs and container state.
- [ ] Document the image tag or digest used for the test.
- [ ] Sketch a CI pipeline that tests, builds, scans, publishes, and deploys the image.

---

17. Docker Mastery Checklist

- [ ] Explain multi-stage builds and layer caching.

- [ ] Build a production-oriented Spring Boot image.
- [ ] Exclude unnecessary build-context files.
- [ ] Explain image security and non-root execution.
- [ ] Understand health checks and restart-policy limitations.
- [ ] Configure resource limits and graceful shutdown.
- [ ] Design a basic CI/CD workflow for container images.
- [ ] Debug startup, memory, and deployment incidents.
- [ ] Explain image tags versus digests.
- [ ] Defend Docker deployment choices in a backend interview.

Final Completion Protocol

Docker's three-chunk curriculum is now authored in this conversation. Once all three source files are saved and verified in the repository:

1. Confirm all three files exist under `content/docker/`.
2. Verify that all three learning layers are covered across the complete handbook.
3. Generate **one final Docker PDF only** at `docs/docker/BEH_DOCKER_COMPLETE_HANDBOOK.pdf`.
4. Verify the PDF's code, diagrams, headings, and page layout.
5. Update README progress using verified status.
6. Commit the completed source and PDF with the exact message:

```
Docker done
```

Do not generate separate chunk PDFs.

Next tool: Redis — Chunk 1 of 3: Caching Fundamentals, Data Structures & Core Mental Models.